NOCR: Vision Language Models for Structured Historical Data Extraction

Auke Rijpma, UU, a.rijpma@uu.nl Rick Schouten, UU

March 23, 2026

Abstract

Extracting structured data from historical archives is a time-consuming bottleneck in economic history research. Traditional approaches require optical character recognition (OCR) followed by manual postprocessing to structure the output. This two-step process is both inefficient and error-prone. This paper demonstrates that vision language models (VLMs) can collapse this pipeline into a single step, directly converting scanned images of historical documents into structured JSON output. This drastically reduces the need for post-processing. Using Google's Gemini Flash models on handwritten Dutch tax records from 1899, we evaluate performance across multiple model versions (2.0, 2.5, and 3.0) and prompting strategies (zero-shot and few-shot). Our results suggest that this VLM-based extraction, which we call NOCR, is already viable for large-scale historical data projects, with clear evidence of rapid model improvement over time.[1]

1. Introduction

1.1 The Challenge of Historical Data Extraction

Economic historians rely on quantitative methods to analyze archival records, from tax assessments and census data to merchant ledgers and notarial records. The digitization of these sources increasingly follows this pipeline: physical documents are scanned, then processed through optical character recognition (OCR) or handwritten text recognition (HTR) systems, and finally the raw text output is postprocessed into structured datasets suitable for analysis.

This two-step approach has significant limitations. Traditional OCR/HTR systems already excel at transcribing text but produce unstructured output that requires extensive manual intervention. As a result, for economic historians working with tabular or otherwise structured data, the postprocessing step can consume more time than the initial transcription. Researchers must write custom parsers, handle implicit information (such as repeated values in table columns), and correct

[1]Thanks to William Skoglund for valuable input. All code to run and explore the experiments is available at https://github.com/HIP-NL/nocr-experiments.git

systematic errors in the OCR output.

1.2 Vision Language Models as a Solution

Recent advances in vision language models (VLMs) offer a compelling alternative: directly converting scanned images into structured data. Rather than extracting text and then inferring structure, VLMs can be prompted to understand both the visual layout and semantic content of a document simultaneously. We call this approach "NOCR", because we see it as a significant departure from traditional OCR methods. This approach even allows VLMs to handle complex cases that traditional OCR pipelines struggle with. Examples include cells that implicitly copy values from rows above, or annotations written in margins.

The key advantage is simplicity: a single API call replaces a multi-stage pipeline. Instead of maintaining separate OCR and postprocessing code, researchers can describe their desired output format in natural language and receive structured JSON in response, which can easily be converted to tabular data use in economic historians' workflows. This approach is particularly well-suited to historical documents, where the visual context often matters as much as the raw text.

We have used this approach to extract the tax data in the HIP-NL project (Rijpma et al. 2025).[2]. It has handled 10000s records as very low cost, including typewritten, printed and handwritten records.

1.3 This Study

We demonstrate the viability of VLM-based extraction using Google's Gemini Flash family of models (Gemini Team et al. 2023) applied to handwritten Dutch tax records from 1899. Our evaluation addresses three questions:

1. Accuracy: How well do VLMs perform at combined OCR and structuring tasks on historical documents?
2. Cost: What are the cost-performance tradeoffs across different models and prompting strategies?
3. Trajectory: How rapidly are these models improving, and what does this suggest for future applications?

Five model variants are tested (ranging from lightweight "flash-lite" models to the latest "flash-3" release), two prompting strategies (zero-shot and few-shot with examples), and extended reasoning modes. The dataset consists of four scanned pages containing 40 individual tax records with ground truth annotations. The results suggest that VLM-based extraction is not only viable but often superior to traditional approaches, particularly when total workflow time is considered.

[2] https://www.hipnl.nl/

2. Data and Methods

2.1 The Utrecht 1899 Tax Records Dataset

Our test dataset comes from the municipal archives of Utrecht in the Netherlands (Het Utrechts Archief). The records document household income tax assessments for the year 1899, with each entry recording the household head's name, address, tax bracket, and taxes due. The appendix provides an example scan and record. While these records are fairly clear and consistent as far as historical archival material goes, the documents present several challenges typical of historical economic data:

Handwriting variation: Multiple clerks contributed to the records, with varying penmanship quality. Some entries are crisp and clear; others show hurried or idiosyncratic letter formation.

Historical orthography: Names and addresses use 19th-century Dutch spelling conventions and abbreviations unfamiliar to modern systems. Titles like "Wed." (widow), "Mej." (miss), and "geb." (born, indicating maiden name) must be correctly interpreted.

Tabular structure with implicit data: The records use a five-column layout where street names are often omitted when they match the previous row. A correct transcription requires understanding this implicit repetition.

Challenging numerals: Tax amounts use a vertical line as the decimal separator, and distinguishing handwritten '0' from '8' requires careful attention to letter formation (zero as a simple oval versus eight as two stacked loops).

Our ground truth consists of four scanned pages (two double-page spreads, split into left and right images) containing 40 individual records, with data split over five columns. Each record includes nine fields that we want to extract: serial number (volgnummer), title, initials, surname, maiden name, street, house number, tax class, and tax amount. Note that these are to be extracted directly, with little to no postprocessing. Ground truth transcriptions were created by manual review.

2.2 Experimental Design

We evaluated five Gemini Flash models:

- gemini-2.0-flash-lite: Lightweight, lowest cost
- gemini-2.0-flash: Standard 2.0 version
- gemini-2.5-flash-lite: Lightweight 2.5 generation
- gemini-2.5-flash: Standard 2.5 version
- gemini-3-flash: Latest 3.0 generation

For each model, we tested multiple configurations:

Prompting strategies: Zero-shot (prompt and target image only) versus few-shot (three example images with ground truth JSON, followed by target image). Few-shot or in-context learning allows the model to learn formatting conventions and

error patterns from examples, at the cost of longer context windows and higher token usage.[3]

Reasoning modes: Standard inference versus extended thinking (2000-token budget). The thinking mode allows models to generate internal reasoning chains before producing output, potentially improving accuracy on complex tasks. Again, this comes at the cost of higher token usage.

Our prompt provides detailed instructions for each field, including specific guidance on edge cases (e.g., "If a street is omitted because it is the same as the previous row fill in the missing value with the last explicitly stated one"). The prompt was iteratively built to try and catch obvious errors in the output. The full prompt appears in Appendix B.

We use a leave-one-out evaluation design: for few-shot experiments with four total images, we use three as examples and the fourth as the target, rotating through all combinations. This ensures each image is evaluated exactly once per model-strategy combination. The Gemini API is stateless, so there is no contamination of the evaluation data.

2.3 Evaluation Metrics

We measure accuracy at two levels:

Character Error Rate (CER). For each record, we calculate the Optimal String Alignment distance (character-level edit distance, including transpositions) between predicted and ground truth values across all nine fields (Loo 2014), divided by the total number of characters in the ground truth. This gives a fine-grained measure of transcription accuracy.

Cell Error Rate. The share of fields (out of nine per record) that contain any error. A field with a single incorrect character counts the same as a completely wrong field. This metric better reflects the downstream cost of errors, since even small mistakes often require manual correction. It should however be noted that some cell errors are not as consequential as others. For example, a one-character mistake in a first name is arguabnly not that important in a situation where spelling variation between names was not uncommon to begin with.

Cost. We calculate the total cost per page based on Gemini API pricing (as of January 2025), accounting for input tokens, output tokens, and thinking tokens where applicable. Prices range from $0.075 per million input tokens (flash-lite) to $3.00 per million output tokens (flash-3).[4]

All 40 records are evaluated for each model-strategy combination, giving 640 total predictions across our experimental design.

[3]Thanks to Niels Rogge for the idea to do this: https://x.com/NielsRogge/status/1901373186043998525.

[4]https://ai.google.dev/gemini-api/docs/pricing. Unfortunately, the generous free tier of the Gemini API (200 requests, one million tokens per day) is no longer available.

3. Model Performance

3.1 Overall Accuracy

Figure 1 shows character and cell error rates across all model-strategy combinations. Several clear patterns emerge:

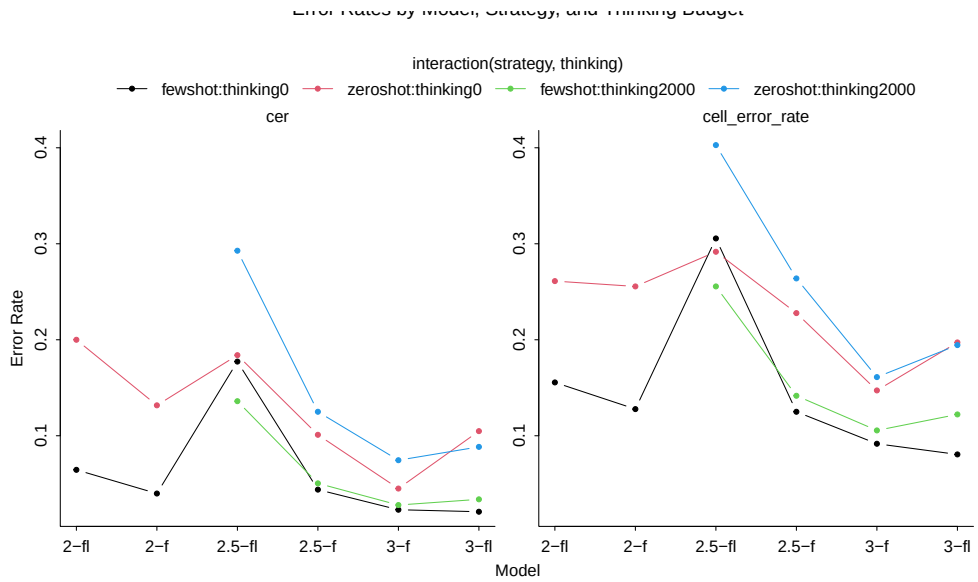


Figure 1: Error rates by model and strategy

The first thing to observe is that the model show substantial improvement over the generations. Gemini 3.0 Flash significantly outperforms earlier versions. Averaged across all strategies, Gemini 3.0 achieves a 4.3% character error rate compared to 8.6% for Gemini 2.0 and 8.0% for Gemini 2.5. At the cell level, Gemini 3.0 achieves 12.6% versus 19.2% for Gemini 2.0 and 19.0% for Gemini 2.5.

Few-shot learning provides substantial benefits. Across all models, few-shot prompting strongly improves error rates. For Gemini 3.0, few-shot reduces CER from 4.5% (zero-shot) to 2.3% (few-shot). The effect is even stronger for earlier model versions—Gemini 2.0's CER drops from 13.2% to 4.0% with few-shot examples. This suggests that showing the model correct formatting is very valuable.

Lite models show varying performance. Sandard models generally outperform their "lite" counterparts Gemini 2.0 Flash Lite achieves 6.5% CER with few-shot prompting—better than Gemini 2.5 Flash zero-shot (10.1%). This has important implications for cost-sensitive applications. At the same time, the 2.5 Flash Lite model performs considerably worse than 2.0 Flash.

Extended thinking shows poor results. The 2000-token thinking budget produces inconsistent effects. For all models except Gemini 2.5 Flash-lite, thinking slightly degrades performance. Notably, Gemini 3.0 often refuses to use extended thinking even when prompted, reverting to standard inference. This suggests that VLMs do not need explicit chain-of-thought to extract data from visual

3.2 Best Performing Configurations

The top five configurations by character error rate are:

model	strategy	thinking	cer	cell_error_rate
gemini-3-flash	fewshot	none	0.023	0.092
gemini-3-flash	fewshot	2000 tokens	0.028	0.106
gemini-2.0-flash	fewshot	none	0.040	0.128
gemini-2.5-flash	fewshot	none	0.044	0.125
gemini-3-flash	zeroshot	none	0.045	0.147

Even the best-performing model makes errors on roughly one in ten fields. Manual inspection reveals these are often challenging cases: ambiguous handwriting (particularly '0' vs '8' and similar letters), unusual abbreviations, or edge cases in the table structure. A 2.3% character error rate translates to roughly one character wrong every 40 characters. This is high enough that output requires review, but in our experience the process is vastly more efficient than manual transcription from scratch.

Interestingly, zero-shot Gemini 3.0 outperforms few-shot Gemini 2.0 and 2.5, suggesting that model quality can substitute for in-context learning to some degree. However, the combination of best model and few-shot prompting remains superior.

3.3 Error Patterns

Examining specific errors reveals predictable patterns. Common mistakes include confusing '0' and '8' in sequential numbers and house numbers (e.g., predicting 1061 vs ground truth 1861), incorrectly handling of implicit street name repetition, and errors in decimal parsing of tax amounts by a vertical

These error types suggest potential improvements through more detailed prompt engineering or fine-tuning on similar documents. The fact that errors cluster around specific challenging cases (rather than being randomly distributed) is encouraging—it suggests that with domain-specific training, even higher accuracy is achievable.

4. Cost-Performance Analysis

4.1 The Cost-Accuracy Frontier

Figure 2 plots error rates against total cost per page for all configurations:

Cost varies dramatically across configurations, from approximately $0.002 per page (Gemini 2.0 Flash Lite zero-shot) to $0.043 per page (Gemini 3.0 Flash few-shot with thinking). This 20-fold range can accommodate different use cases. The cost-accuracy frontier reveals several optima:

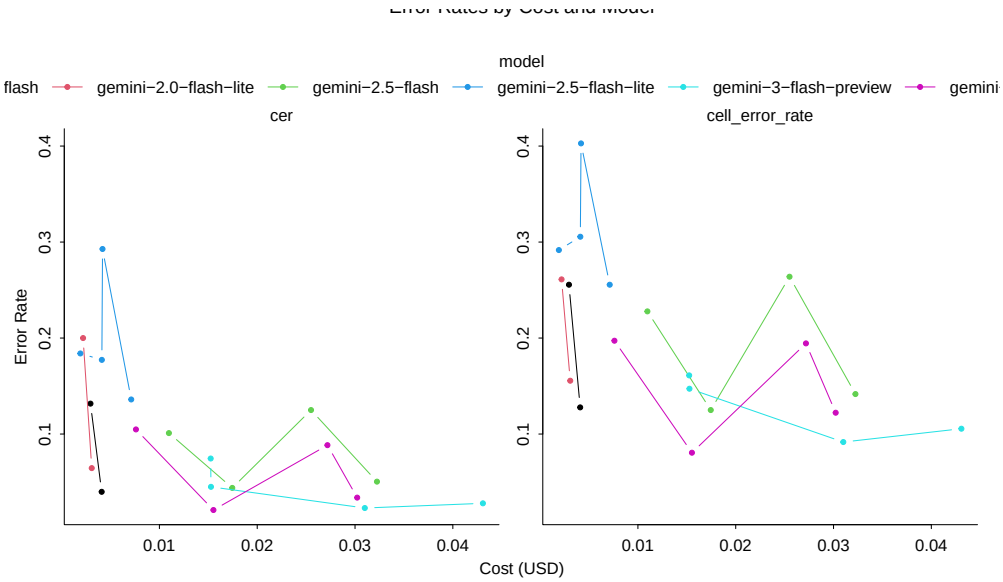


Figure 2: Error rates by cost and model

Budget tier ($0.002–0.003 per page): Gemini 2.0 Flash Lite few-shot achieves 6.5% CER for $0.003 per page. This is the cheapest configuration that breaks 10% CER.

Mid tier ($0.004–0.017 per page): Gemini 2.0 Flash few-shot (4.0% CER, $0.004) and Gemini 2.5 Flash few-shot (4.4% CER, $0.017) offer strong performance at moderate cost.

Premium tier ($0.031 per page): Gemini 3.0 Flash few-shot achieves the best accuracy (2.3% CER) at $0.031 per page. Adding thinking increases cost to $0.043 while slightly degrading performance, making it a suboptimal choice.

For most historical digitization projects, the mid-tier options offer the best balance. At $0.004 per page and 4.0% CER, Gemini 2.0 Flash few-shot would cost $4,000 to process 1 million pages—a fraction of manual transcription costs, with accuracy sufficient for most quantitative analyses. To put those million pages into perspective, the full 1899 tax records are 1108 scans. By 1920, that number had grown to 2828.

4.2 Scaling Considerations

Several factors could further reduce costs at scale:

First, context caching could reduce costs. If the few-shot examples remain constant across pages from the same document type. Gemini's context caching feature could reduce input token costs by up to 75% for few-shot scenarios, making the premium tier more affordable.[5] Because we varied the examples per prompt in these experiments, we could not use this feature here.

Likewise, processing documents in batches could enable further optimizations and

[5] https://ai.google.dev/gemini-api/docs/caching?lang=python.

volume discounts. This reduces costs by 50%. It is straightforward to set up, and the only downside is a delay up to 24 hours.

Mixed strategies could also be valuable. Projects could use expensive models for challenging documents and cheaper models for routine cases, optimizing the overall cost-accuracy tradeoff. This would require a reliable method to to assess model difficulty.

Finally, it is worth discussing VLM pricing. While there have been subsantial reductions, e.g. the costs of 2.5 at its introduction was a factor ten lower than 2.0 at its introduction. Moverover, the overall trend in LLM pricing is negative, decreasing by c. 25% each year. At the same time, if we omit the introduction pricing of 2.5 compared to 2.0, each Gemini Flash model was more expensive than the one that came before.[6] Moreover, the generous free tier of the Gemini API has been removed.

5. Discussion

Our results demonstrate that VLM-based extraction of structured historical data is not just viable but competitive with traditional approaches. The most important finding is that direct scan-to-JSON works reliably. Even complex tabular layouts with implicit data and historical handwriting can be accurately *and directly* parsed into structured output without separate OCR and postprocessing steps.

Performance improves rapidly. The 65% reduction in error rates from Gemini-Flash 2.0 to Gemini-Flash 3.0 (released less than a year apart) suggests we are on a steep improvement curve.

Few-shot learning is highly effective way to improve performance. The dramatic improvement from showing just three examples suggests that in-context learning remains valuable even as base model capabilities improve.

Cost-performance tradeoffs are also favorable. For $0.004 per page, we achieve error rates low enough to support quantitative analysis with modest manual review. This is orders of magnitude cheaper than manual transcription ($1–5 per page for professional transcribers).

There are caveats to our findings. We evaluate only one document type (Utrecht tax records from one year). Generalization to other formats, languages, or time periods requires further testing. Moreover, while 40 records is sufficient for comparing models, it is insufficient for characterizing rare error types or building detailed error models.

Open source models are a promising avenue for further research. Many open VLMs are available that can be run locally (e.g. Qwen2-VL, Wang et al. 2024; PaliGemma Beyer et al. 2024). While often less accurate than commercial models offered over APIs, fine-tuning on historical documents could close the gap while eliminating

[6] https://github.com/sanando/llmpricing. No scholarly review of LLM pricing trends could be found at the time of writing.

API costs and data privacy concerns. It would also avoid vendor lock-in, model deprecation, or pricing changes could disrupt projects mid-stream. Our experiments use general-purpose models. Fine-tuning on large corpora of historical documents (perhaps using cheaper models to generate noisy labels for expensive models to refine) could substantially improve accuracy on domain-specific challenges like historical orthography or archaic abbreviations. Our own experiments in finetuning the DONUT family of models were promising in this regard (Kim et al. 2022).

Getting models to provide confidence scores for each extracted field would also be valuable. This would enable a hybrid workflows where high-confidence extractions are accepted automatically while uncertain cases receive manual review, or follow-up with a more powerful and expensive model.

Finally, the tabular tax records used represent one type of document in the vast space of historical records available in the archives. Applying similar methods to census records, notarial archives, merchant ledgers, and correspondence would demonstrate broader applicability.

The viability of VLM-based extraction has significant implications for historical research. As costs fall and accuracy improves, creating structured datasets from archival sources becomes feasible for individual researchers with limited funding. Projects involving millions of pages become tractable. The entire Utrecht tax series (hundreds of volumes) could be digitized for a few thousand dollars rather than hundreds of thousands. The key constrain, rather, is whether archives can provide their records as high-quality scans.

Appendix A: Sample Scans and Ground Truth

The scans containing the tax records look as follows:

Below is a sample ground truth record demonstrating the structure and complexity of the data:

```
{
    "volgnummer": 1861,
    "title": null,
    "initials": "G.",
    "surname": "Fukkink",
    "maiden_name": null,
    "street": "Oudekamp",
    "house_number": "10",
    "class": 2,
    "tax": 4.5
}
```

This record shows a typical entry: sequential number 1861, no title, initials "G.", surname "Fukkink" (note the challenging double-k and -ink ending), address at Oudekamp 10, tax class 2, and tax amount of 4.5 guilders (using decimal notation after conversion from the vertical-line separator in the original).

More complex cases include widows with maiden names:

```
{
    "volgnummer": 1870,
    "title": "Wed.",
    "initials": "H.Th.E.",
    "surname": "Kuijper",
    "maiden_name": "J.M. Leuven",
    "street": "Oudkerkhof",
    "house_number": "5",
    "class": 1,
    "tax": 1.5
}
```

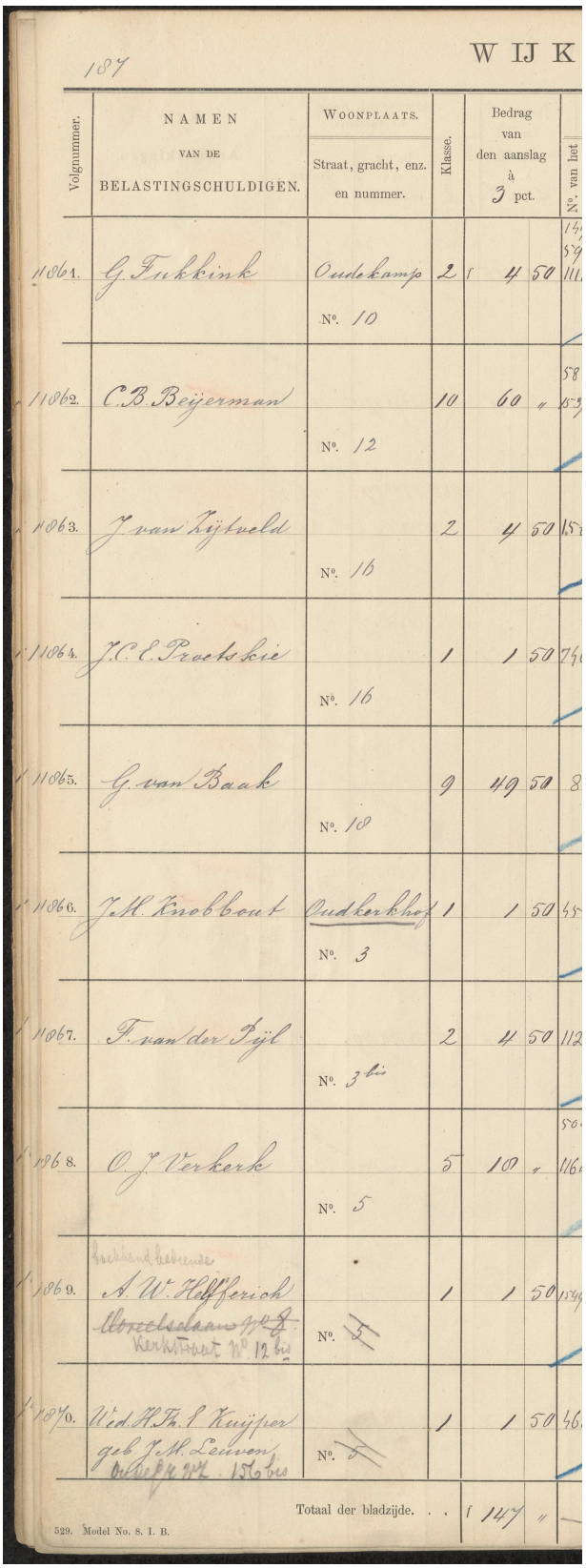
Here "Wed." (Weduwe, widow) as the title, multi-part initials "H.Th.E.", and the maiden name "J.M. Leuven" (indicated by "geb." in the original scan) demonstrate the richer structure that VLMs must correctly parse.

Appendix B: Prompt Engineering

The full prompt used for all experiments:

```
Perform the following task using step-by-step reasoning.

Extract the data from this scan and return it as json.
```



187

WIJK

Volgnummer.	NAMEN VAN DE BELASTINGSCHULDIGEN.	WOONPLAATS. Straat, gracht, enz. en nummer.	Klasse.	Bedrag van den aanslag à 3 pct.		N°. van het
11061.	G. Tukkink	Oudekamp N°. 10	2	4	50	15 59 111
11062.	C.B. Beijerman	N°. 12	10	60	"	58 159
11063.	J. van Zijtveld	N°. 16	2	4	50	158
11064.	J.C.E. Proetskie	N°. 16	1	1	50	74
11065.	G. van Baak	N°. 18	9	49	50	8
11066.	J.H. Knobbout	Oudkerkhof N°. 3	1	1	50	45
11067.	F. van der Pijl	N°. 3 bis	2	4	50	112
11068.	O.J. Verkerk	N°. 5	5	10	"	50 116
11069.	Boekhoud. bedreven A.W. Hoffkerich Utrechtsdraaw N°. 8 Kerkstraat N°. 12 bis	N°. 5	1	1	50	134
11070.	Wed. H.Th.E. Kuijper geb. J.M. Leeuwen Oudegracht N°. 156 bis	N°. 5	1	1	50	46
		Totaal der bladzijde. . .		147	"	

529. Model No. S. I. B.

Figure 3: Handwritten municipal income tax records from the Utrecht Archives, 1007-2 Gemeentebestuur van Utrecht 1813-1969, no 6613

The scan contains five columns describing individuals being taxed in the Dutch city of Utrecht in 1899. Extract the following information and return it as json:

- volgnummer, the sequential number of the individual in the scan, an integer number at most four digits long. If there are more digits, the first one is probably from the previous page and should be ignored.

- title, a title such as 'Wed.', 'Dr.', 'Mej', 'jkhr', 'jkvr', 'geb', 'vert', 'p/a', 'cur'. If the scan does not contain a title, leave this empty (null).

- initials, the initials of the individual.

- surname, the surname of the individual.

- maiden name, the full maiden name of the individual, below the husband's name and indicated with 'geb' (for 'geboren'), typically provided if she is a widow.

- street, the street in Utrecht of the individual, always in the 'woonplaats' column. If a street is omitted because it is the same as the previous row fill in the missing value with the last explicitly stated one. If the scan does not start with a street to fill in, leave the street empty. Below the name another address may be written if the individual moved. You should ignore this and only provide the original address from the woonplaats column.

- house_number, the house number of the individual, after the printed 'No.'. This is an integer number, sometimes followed by a letter (a, b), a supers

- class, the tax class of the individual, in the 'klasse' column', an integer number. This should always be transcribed. In the first row there will be an 'f' next to the class, which should be ignored.

- tax, the tax amount due for the individual, in the 'Bedrag van den aanslag' column. This is a decimal number, where a vertical printed line is used as the decimal separator. A quote mark can also follow the vertical line instead of the usual two digits, which should be transcribed as two zeroes. A little bit of the next column might be visible, but this should be ignored, only the part in the 'Bedrag van den aanslag' column should be transcribed.

Carefully distinguish between the digits '0' and '8'. The digit '0' is

```
a simple oval with a continuous outline, maybe open at the top if the
writer was in a hurry, while '8' consists of two stacked loops, typically
crossing the loop.
```

The prompt combines general instructions with specific edge case handling. Key design choices include:

1. Explicit field descriptions: Each JSON field is described in detail with examples
2. Edge case guidance: Handling implicit data (street repetition), moved addresses, and formatting conventions
3. Character-specific instructions: The '0' vs '8' distinction required specific guidance after initial experiments revealed this as a common error
4. Context about document structure: Explaining column headers and layouts helps the model understand the visual organization
5. Output format specification: Requesting JSON ensures structured, parsable output

The "step-by-step reasoning" prefix was included to encourage extended thinking, though as noted earlier, Gemini 3.0 often ignores this instruction when it deems reasoning unnecessary.

References

Beyer, Lucas, Andreas Steiner, André Susano Pinto, et al. 2024. *PaliGemma: A Versatile 3B VLM for Transfer*. arXiv:2407.07726. arXiv. http://arxiv.org/abs/2407.07726.

Gemini Team, Rohan Anil, Sebastian Borgeaud, et al. 2023. *Gemini: A Family of Highly Capable Multimodal Models*. arXiv. https://doi.org/10.48550/ARXIV.2312.11805.

Kim, Geewook, Teakgyu Hong, Moonbin Yim, et al. 2022. "OCR-Free Document Understanding Transformer." In *Computer Vision – ECCV 2022*, edited by Shai Avidan, Gabriel Brostow, Moustapha Cissé, Giovanni Maria Farinella, and Tal Hassner, vol. 13688. Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-19815-1_29.

Loo, Mark van der. 2014. "The Stringdist Package for Approximate String Matching." *The R Journal* 6 (1): 111–22. http://CRAN.R-project.org/package=stringdist.

Rijpma, Auke, Eva van der Heijden, Alex Nagel, Paul Puschmann, and Rick Schouten. 2025. *An Historical Income Panel for the Netherlands, 1850–1920*.

Wang, Peng, Shuai Bai, Sinan Tan, et al. 2024. *Qwen2-VL: Enhancing Vision-Language Model's Perception of the World at Any Resolution*. arXiv:2409.12191.

arXiv. https://doi.org/10.48550/arXiv.2409.12191.